Available online at www.sciencedirect.com

ScienceDirect

IFAC-PapersOnLine 49-12 (2016) 017–022

A Dynamic Programming Algorithm for the Online Cutting Problem with Defects and Quality Grades

Dilek Tuzun Aksu*. Bahadır Durak .*

**Yeditepe University Department of Industrial and Systems Engineering, Ataşehir, Istanbul, 34755 TURKEY (Tel: 90-216-578-0417; e-mail: dtuzun@yeditepe.edu.tr, bahadir_durak@yahoo.com).*

Abstract: In this article, we propose a dynamic programming based algorithm for an online glass cutting problem that arises in the glass industry. In flat glass production, a continuous ribbon of flat glass is cut into glass products of various sizes as it flows on a conveyor belt. The glass ribbon contains defects of different types based on their severity and each glass product belongs to a quality class that indicates the maximum number of defects of each type that can be tolerated. Since the location and type of defects are detected in real time using a camera, the cutting problem has to be solved online and each cutting decision is made within a few seconds. The primary goal of this cutting problem is to minimize the amount of leftover (*scrap*) glass resulting from the cut products. The proposed algorithm uses a look-ahead strategy to solve a static cutting problem at each iteration to determine the products to be cut within a fixed horizon length, but implements only the first one or few of these cuts per iteration. We evaluated the algorithm on a set of realistic problems reflecting the parameters of current flat glass production lines. Computational results indicate that the proposed algorithm can reduce the amount of scrap glass by an average of 26.1%.

© 2016, IFAC (International Federation of Automatic Control) Hosting by Elsevier Ltd. All rights reserved.

Keywords: cutting stock problems, dynamic programming, flat glass production, defects, online algorithm.

1. INTRODUCTION

In this article, we present an online algorithm for a cutting problem that arises in flat glass production. In a flat glass production line, which is also called a *float* line, molten glass from the furnace is formed into a ribbon that cools down and solidifies as it moves on the conveyor belt. Rectangular glass products of various sizes are then cut from this flat glass ribbon while the conveyor is moving at a constant speed. During the production process, defects of varying severity levels form on the glass ribbon. On the other hand, each glass product to be cut from the glass ribbon belongs to a quality class that specifies the maximum number of defects of each type that are permitted on the product. A product that is cut from a piece of glass that does not meet the requirements of its quality class has to be discarded as scrap. Scrap glass is sent back to the furnace to be melted and recycled, which results in increased production costs as well as reduced productivity. Thus, the primary goal of the cutting process is to minimize the amount of scrap glass resulting from the cuts. Maximization of the total value obtained from cut products is also taken into consideration in this process. The camera that identifies the coordinates and type of the defects is located only 3-4 seconds before the cutting bridges that score the cuts on the ribbon. Therefore, the time to make the cutting decisions is quite limited. Also, at the time of decision making, the length of the glass ribbon with defect data usually does not exceed 12-15 meters. Therefore, cutting decisions need to be made with limited data and within a very short timeframe. In this work, we propose an online dynamic programming based algorithm to determine the type and location of the products to be cut from the glass ribbon in real time. The algorithm solves a series of static cutting problems over partially overlapping lengths of glass ribbon.

In the literature, the two dimensional glass cutting problem described here is classified as a two-stage problem since the cuts are scored on the glass ribbon in at most two stages: an x-cut that cuts along the width of the glass ribbon end-to-end, (possibly) followed by a y-cut along the length of the ribbon that cuts the rectangle formed by the x-cut into smaller rectangles. The cuts are scored by bridges located over the glass ribbon. x-bridges use one cutting wheel to score the glass ribbon end-to-end while the y-bridges have cutting wheels at up to 3-5 positions along the y-bridge to enable simultaneous application of multiple y-cuts. The fixed number of cutting wheels on the y-bridges limits the number of product patterns or *configurations* that can be placed along the width of the ribbon. Usually the number of different configurations that can be cut on the float line does not exceed 10-15. We take advantage of this limited number configurations to reduce the two-dimensional problem over product types to a one dimensional problem over product configurations. An important feature of flat glass cutting process which separates it from many other cutting problems is that it allows strips of defective glass to be cut between configurations. These glass strips are recycled as scrap glass in an effort to avoid anticipated quality problems in later cuts. The industry standard is to perform such cuts immediately behind a serious defect, therefore they are known as Cut Behind Fault (CBF) cuts in float line terminology. Fig. 1 displays a glass ribbon on which a cutting pattern that includes three different configurations and one CBF is marked. The glass ribbon contains defects of two quality grades that are marked as small

2405-8963 © 2016, IFAC (International Federation of Automatic Control) Hosting by Elsevier Ltd. All rights reserved.
Peer review under responsibility of International Federation of Automatic Control.
10.1016/j.ifacol.2016.07.543

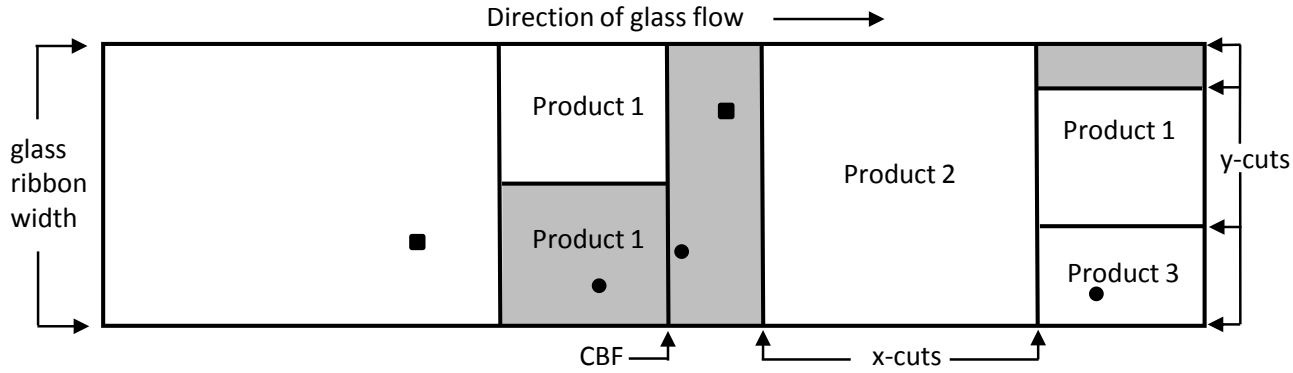

Fig. 1. Illustration of a cutting pattern placed on the glass ribbon on a float line

circles and squares. Three types of products (Products 1-3) that are cut using x- and y-cuts are marked with vertical and horizontal lines. Shaded areas represent the scrap glass that results from the placement. Products 1-3 are used to form three configurations consisting of one or two products each. The first of these configurations includes one Product 1 and one Product 3 as well as some scrap glass. Please note that this configuration can be cut using four cutting wheels on the y-bridge. The second configuration contains only one Product 2 followed by a CBF after the circle type defect. The last configuration consists of two units Product 1, one of which is discarded as scrap due to the circle type defect on it. Note that the same grade defect does not render Product 3 scrap since the two products belong to different quality grades: Product 3's quality grade allows a circle defect while Product 1's does not. One should also note that the last configuration requires another cutting wheel positioned in the middle of the glass ribbon. Therefore the three configurations displayed in Fig. 1 requires a total of five cutting wheels located at different positions on the y-bridge.

2. RELATED LITERATURE

Cutting problems have been studied extensively in the literature. One can refer to one of the various surveys available for an overview on the topic. (Dowsland and Dowsland (1992), Dyckhoff (1990), Bischoff and Wäscher (1995), Wäscher, et al. (2007)). Despite the large body of literature on cutting problems, studies on cutting problems with quality variations and defects is quite limited. One of first such studies that focus on quality variations is due to Gilmore and Gomory (1965). Here, the authors study the one and two dimensional versions of the static cutting stock problem where the value of the cut product depends on its position and propose a recursive algorithm to solve both versions. In a later study, Hahn (1968) defines a cutting stock problem with guillotine cuts and defects. In this problem, the products are not allowed to be placed on areas that contain defects. A dynamic programming method suggested in Gilmore and Gomory (1966) is used to solve this problem. Scheithauer and Terno (1988) also solve a more general version of the problem in Hahn (1968) where the cutting stock is not restricted to be a rectangle using dynamic programming. A more recent study by Gelder and Wagelmans (2009) focuses on a similar problem in the area of roller blind production and solves it using a two stage heuristic. Afsharian et al. (2014) also employ a dynamic programming based heuristic to solve a two dimensional cutting problem with defective areas that can be approximated by rectangular regions. There are also other studies on cutting problems where the cutting stock contains a single defect e.g. Carnieri et al. (1993), Neidlein et al. (2009).

There are several articles in the literature on one dimensional cutting problems where defects and/or quality grades are considered. For instance, Sculli (1981) defines a problem where small rolls of fixed size are cut from a long roll that contains defects. In this variant, size of the small rolls is a random variable due to the defects on the long roll. In another cutting problem from the timber industry, Faaland and Briggs (1984) define the process of cutting a tree into logs and cutting these logs further into lumber. In this problem, shape and quality variations on the tree require a specialized approach. In Sarker (1988), the cutting stock contains several punctual defects and the objective is to attain the maximum total value by cutting only through defects. In this variant, the value of each resulting piece depends on its length and the number of defects on it. Sweeney and Haessler (1990) discuss a one dimensional problem, in which the cutting stock contains various sections of hierarchical quality grades and customers also specify minimum acceptable quality grades for their orders. Aboudi and Barcia (1998) consider a variant taken from the paper industry. In this problem, a roll of paper containing a single defective area is cut vertically to produce a set of sheets. Since the sheets that contain a defective area are to be discarded, objective is to find a cutting plan that minimizes the total length of the defective sheets. In an application from the textile industry, Özdamar (2000) proposes a simulated annealing algorithm to create a cutting and wrapping solution where a cutting stock with multiple defects is first cut into smaller pieces and each piece is assigned a quality grade based on the defects it contains. Next, pieces with the same quality grade are wrapped to create rolls of given quality specifications. Although each of these problems bears similarities with the problem defined in this article, they all differ from the glass cutting problem in terms the way the defects, quality classes, and the conditions for creating feasible cuts. More importantly, all of the mentioned problems are static in nature, as opposed to the problem at

hand, where the defect locations and types become available in real time.

There are very few studies regarding online cutting problems in the literature. To our knowledge, the only work that discusses on an online problem with defects is by Ghodsi and Sassani (2005). In their study, the authors describe an online cutting problem from solid wood furniture production in which the stock (i.e. wood strip) size varies. Each wood strip has both defects that need to be removed and quality variations along the length of the strip. There are important differences between the problem in Ghodsi and Sassani (2005)'s paper and the one defined here in the way defects and quality are treated: In their version, defects have to be removed completely by guillotine cuts applied along the width of the strip. Then, resulting non-defective areas are separated into quality intervals such that within a quality interval the entire area has the same quality grade along the width of the strip. This reduces the problem to one dimension. In our problem, we consider point defects that only affect the items placed on the area containing the defect (i.e. the item has to overlap with the defect on both dimensions), which does not allow us to reduce the problem to one dimension entirely. Also, in Ghodsi and Sassani (2005), quality grades of the strip and the quality grades of the items are the same, whereas in our version, there is no one-to-one correspondence between defect grades and the quality grades of each product. As discussed earlier, in the glass cutting problem, quality grade of a product is defined as the maximum number of defects of each type that are allowed on that product type.

Our review of the related literature indicates that the glass cutting problem described here differs from the previous studies not only in the way that defects and quality specifications are defined but also in terms of its online nature. Technical specifications of the glass production line require that the cutting decisions are made in a matter of seconds, which makes this problem computationally challenging. In the next section, we introduce the online glass cutting algorithm that utilizes a dynamic programming approach to solve the glass cutting problem efficiently.

3. SOLUTION APPROACH

3.1 Online Glass Cutting Algorithm

In this study, we propose an algorithm that solves the online glass cutting problem on a rolling horizon basis by decomposing it into a series of static cutting problems solved over overlapping parts of the glass ribbon of a fixed length, L. (see Fig.2) In each iteration of the online algorithm, a static cutting problem is solved using a dynamic programming procedure to determine the configurations to be placed on the fixed length of glass. While the problem is solved over the length L, only the first few (n) of these configurations are added to the cutting plan for execution. In the next iteration, a new cutting problem is solved for the next L meters immediately following the last configuration on the cutting plan. The value of n is chosen as a small integer, usually 1 or 2. Using a small value for n results in implementing only a small part of the static solution at each iteration. The rest of the static solution is computed to avoid making myopic decisions.

The solution approach currently used in float lines is similar to the look-ahead algorithm depicted in Fig. 2. However, instead of solving a static problem over a constant length L at each iteration, the current practice uses explicit enumeration to identify the best solution among all possible permutations of a fixed number (m) of consecutive configurations and implement the first configuration from the selected permutation. During the enumeration, placing a CBF in any of these m positions is also evaluated. Therefore, the algorithm is capable of evaluating solutions that contain one or more CBFs. Unfortunately, the value of m is usually set to a small integer (such as 3) due to the computational inefficiency of explicit enumeration, which limits the look-ahead capability of the online algorithm and causes it to produce inferior solutions.

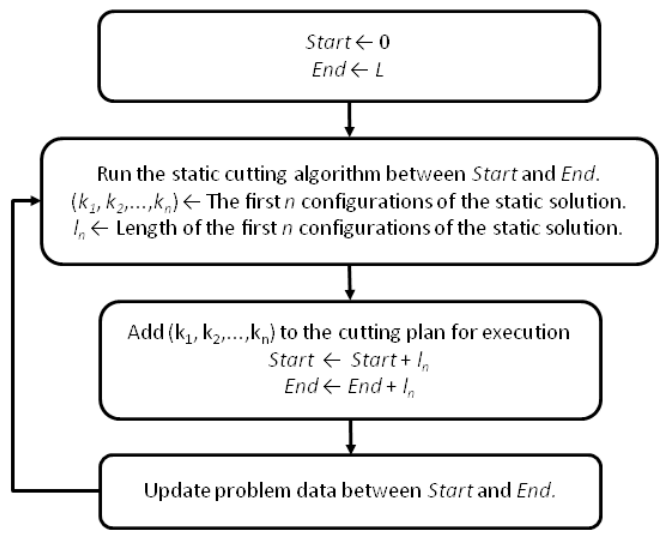

Fig. 2. Online glass cutting algorithm flowchart

3.2 Dynamic Programming Procedure for the Static Glass Cutting Problem

We propose a dynamic programming (DP) procedure to solve the static one dimensional cutting problem at each iteration of the glass cutting algorithm. This algorithm is inspired by the approach described by Gilmore and Gomory (1965) for a cutting stock problem in which the value of each item is dependent on its position on the stock. However, our version is significantly different than the one described in this article in the way it accounts for multiple point defects and allows for the usage of CBF. The procedure outlined below finds the optimal solution for the static cutting problem over a glass ribbon of length L in which removal of scrap glass between configurations is allowed in the form of one or more CBFs. The procedure calculates the best sequence of configurations and CBF's to be located on the glass ribbon of length L. The state of the DP procedure is the x-coordinate of the point (between 0 and L) for which a cutting decision is to be made. The optimal objective function value $v(L)$ is computed using

the recursive procedure below. Before describing the procedure, we define the necessary notation:

L : length of the glass ribbon to be cut (glass ribbon is positioned between x-coordinates $0 - L$),

N : number of all possible product configurations that can be obtained by the cutting wheels positioned on the y-bridge,

D : number of defects on the glass ribbon,

x_d : x-coordinate of defect d on the glass ribbon, $d = 1,2,\dots,D$,

l_i : length of configuration i along the x-axis, $i = 1,2,\dots,N$,

$\pi_i(x)$: value of configuration i when it is placed between x-coordinates x and $x - l_i$, $i = 1,2,\dots,N$,

$\mu(x)$: value of scrap glass of length x (Width of the scrap glass is W and the value is independent of the position of the scrap glass on the glass ribbon),

$\delta(x)$: the distance (along the x-axis) between point x and the first defect after point x in the direction of glass flow, i.e.:

$$\delta(x) = \begin{cases} x - \max\limits_{d:\, x_d \le x} x_d & \exists d:\ x_d \le x \\ x & \nexists d:\ x_d \le x \end{cases} \tag{1}$$

$v(x)$: the maximum value that can be obtained for the portion of the glass ribbon that lies between coordinates 0 and x.

Note that $\pi_i(x)$, i.e. the objective function contribution of configuration i placed at coordinate x should be calculated based on the position of the products in configuration i and the coordinates of defects between x-coordinates $x - l_i$ and x. The value of $\mu(x)$, i.e. the objective function contribution of a piece of scrap glass of length x and width W can be assumed 0, or any value deemed appropriate by the decision maker. If the decision maker deems scrap to be undesirable, its value can be assigned a high negative value to discourage its usage. In general, the values of $\pi_i(x)$ and $\mu(x)$ should be set based on the importance of the two objectives stated previously (scrap area minimization and product value maximization) for the decision maker.

Dynamic Programming Procedure for the Static Glass Cutting Problem

Step 1. Initialization

1.a) List all configurations i that can be obtained using available cutting wheels on the y-bridge, ($i = 1,2,\dots,N$).

1.b) Calculate $l_i\ \forall i = 1,2,\dots,N$.

1.c) Initialize $v(x)\ \forall x < \min\limits_i\{l_i\}$:

$$v(x) = \mu(x) \qquad \forall x < \min_i\{l_i\} \tag{2}$$

Step 2. Recursion

Calculate $v(L)$ using the recursive function below:

$$v(x) = \max \begin{Bmatrix} \max\limits_{i:x \ge l_i}\{\pi_i(x) + v(x - l_i)\} \\ \mu(\delta(x)) + v(x - \delta(x)) \end{Bmatrix} \quad \forall x \ge \min_i\{l_i\} \tag{3}$$

The function $v(x)$ calculates the maximum value that can be achieved for the glass ribbon between coordinates 0 and x by placing either a configuration or a CBF. The first argument in (3) calculates the maximum value that can be obtained by placing any configuration i between coordinates x and $x - l_i$ of the glass ribbon. The second argument evaluates the value that can be obtained by placing a cut behind the next defect after coordinate x (i.e. a cut behind fault, or CBF performed after x). The recursive equation in (3) finds the maximum value that can be attained by either placing a configuration or performing a CBF after point x by taking the maximum over both arguments.

4. COMPUTATIONAL RESULTS

In order to test the performance of the online glass cutting algorithm, we conducted a computational experiment on five realistic problem instances, each of which consists of a glass ribbon that is 3.21m. wide and 1200m. long with uniformly distributed defects at a density of 0.83 defects/m^2. We should note that we collaborated with Em-glass, a company that specializes in the design of flat glass production lines, to ensure that the experiment is performed on realistic data. We experimented with two main parameters that impact solution quality: the length of the glass ribbon used for the static problem (L) and the number of configurations implemented from each static solution (n). L is set at three levels: 6m., 9m., and 12m. n is set as 1, 2, and n_{max}, where n_{max} stands for implementing all of the configurations obtained from the static solution. In order to compare the performance of the algorithm with the current practice, we also coded the explicit enumeration (EE) approach used in the industry and ran it using a setting of m=3 in line with the current practice. Both the DP and the EE approaches were implemented in MATLAB® (R2009b). All runs that are performed in the computational experiment were conducted on an average notebook computer with Intel(R) Core(TM) i7 CPU at 2.40 GHz and 12 GB RAM.

Table 1 summarizes the computational performance of the DP based online glass cutting algorithm (denoted by DP) in comparison with the explicit enumeration (EE) approach used in the industry. In this table, the first row displays the method used followed by the parameter settings for L and n for the DP algorithm. The next five columns give the objective function values obtained by the indicated method and parameter settings for the five problem instances. The second to last column provides the average objective function value over the five instances. Finally, the last column gives the average CPU time (in seconds) per static problem solved throughout the online algorithm. The objective function value displayed for each problem instance is the total area of scrap glass generated over the 1200m. long glass ribbon. Since the primary goal of flat glass producers is scrap minimization, we concentrated on minimization of scrap glass for the computational study. However, using a combination of product value and scrap by appropriately setting the values of $\pi_i(x)$ and $\mu(x)$ is equally

straightforward computationally as we discussed in the previous section.

Based on the reported results, it is clear that the proposed DP based algorithm is superior to the widely used EE approach used in the industry. The results also indicate that the scrap reduction is achieved by using a lower setting of n and a higher setting of L. In fact the best results are obtained for all five instances for n=1 and L=12. A lower value of n indicates that only a small number of configurations from the current static solution is implemented, which corresponds to delaying the cutting decisions so that better decisions can be made based on the defect data that becomes available later. On the other hand, a higher value of L means that each static problem considers a long planning horizon, allowing for a less myopic decision. Note that the average time to compute each static solution does not exceed the 3 second limit for any of the DP parameter settings. However, our experiments indicate that the CPU time to solve a small percentage of the static problems exceeds 3 seconds when L is set to 12m. Although these instances are rare and the performance issue can be overcome by a more powerful industrial quality computer, we take the more conservative approach and suggest the setting of n=1 and L=9, which produces good quality results and never violates the 3 second limit per static problem. For this setting, the DP based algorithm is still an average of 26.1% better than the EE approach.

Table 1. Objective function value (total area of scrap glass) obtained by the EE and DP techniques in m^2.

Met	L	n	1	2	3	4	5	Obj.	CPU
DP	6	1	775	741	791	754	753	**763**	**0.29**
		2	806	754	805	750	765	**776**	**0.29**
		n_{max}	861	841	888	835	846	**854**	**0.09**
	9	1	766	721	774	728	744	**747**	**0.97**
		2	774	745	773	738	747	**755**	**0.95**
		n_{max}	828	787	839	802	811	**814**	**0.44**
	12	1	743	706	749	706	715	**724**	**2.53**
		2	743	719	755	714	728	**732**	**2.44**
		n_{max}	793	775	798	766	768	**780**	**1.31**
EE	n/a		1044	1004	1016	1011	977	**1010**	**0.44**

4. CONCLUSION

In the glass industry, reduction of scrap glass is critical in cutting production costs and staying competitive. The proposed algorithm provides an efficient algorithm that is proven to achieve significant savings for glass manufacturers. One avenue of further research is to investigate the impact of the CBF assumption for scrap cuts. While this assumption enables us to solve the static cutting problem within seconds, allowing scrap cuts at arbitrary locations (not just behind defects) may improve solution quality. Unfortunately, without the CBF assumption, solving the static cutting problem to optimality is not straightforward. We plan to focus on this more general problem next. Although the literature on one and two dimensional cutting problems is quite extensive, cutting problems with defects and quality grades have not been studied widely. This study focuses on a problem that arises in the glass industry, however the proposed approach can also be adapted to cutting problems in other industries such as lumber, steel, and fabric. We also plan to explore cutting problems with defects in other industries as well.

ACKNOWLEDGEMENTS

This research was supported by the Scientific and Technological Research Council of Turkey (TÜBİTAK) under Grant 114M372. We would also like to thank Mr. Hüsnü Evren and Mr. Gürkan Balcı from Em-glass for their continuous support throughout the study.

REFERENCES

Aboudi, R. and Barcia, P. (1998). Determining cutting stock patterns when defects are present. *Annals of Operations Research*, 82, 343-354.

Afsharian, M., Niknejad, A., and Wäscher, G. (2014). A heuristic, dynamic programming-based approach for a two-dimensional cutting problem with defects. *OR Spectrum*, 36, 971-999.

Bischoff, E. and Wäscher, G. (1995). Cutting and packing: Special Issue. *European Journal of Operational Research*, 84, 503-505.

Carnieri, C., Mendoza, G.A., and Luppold, W.G. (1993). Optimal cutting of dimension parts from lumber with a defect: A heuristic solution procedure. *Forest Products Journal*, 43(9), 66-72.

Dowsland, K.A. and Dowsland, W.B. (1992). Packing problems. *European Journal of Operational Research*, 56, 2-14.

Dyckhoff, H. (1990). A typology of cutting and packing problems. *European Journal of Operational Research*, 44, 145-159.

Faaland, B., and Briggs, D. (1984). Log bucking and lumber manufacturing using dynamic programming. *Management Science*, 30, 245-257.

Gelder, E.R. and Wagelmans, A.P.M. (2009). The two-dimensional cutting stock problem within the roller blind production process. *Statistica Neerlandica*, 63(4), 474-489.

Ghodsi, R. and Sassani, F. (2005). Real-time optimum sequencing of wood cutting process. *International Journal of Production Research*, 43(6), 1127–1141.

Gilmore , P.C. and Gomory, R.E. (1965). Multistage cutting stock problems of two or more dimensions. *Operations Research*, 13, 94-120.

Gilmore , P.C. and Gomory, R.E. (1966). The theory and computation of knapsack functions. *Operations Research*, 14, 1045-1074.

Hahn, S.G. (1968). On the optimal cutting of defective sheets. *Operations Research*, 16, 1100-1114.

MATLAB® Version 7.9.0.529 (R2009b). The MathWorks, Inc., Natick, Massachusetts <www.mathworks.com> .

Neidlein , V., Vianna, A.C.G., Arenales, M.N., and Wäscher, G.. (2009). "Two-Dimensional Guillotineable-Layout Cutting Problems with a Single Defect - An AND/OR-

Graph Approach." In Fleischmann, B., Borgwardt, K-H., Klein, R., and Tuma, A. (eds.), *Operations Research Proceedings 2008 Part 3*, 85-90. Springer, Berlin-Heidelberg.

Özdamar, L. (2000). The cutting-wrapping problem in the textile industry: Optimal overlap of fabric lengths and defects for maximizing return based on quality. *International Journal of Production Research*, 38(6), 1287-1309.

Sarker, B.R. (1988). An optimal solution for one-dimensional slitting problems: a dynamic programming approach. *Journal of the Operational Research Society*, 39, 749-755.

Scheithauer, G. and Terno, J. (1988). Guillotine cutting of defective boards. Optimization, 19, 111-121.

Sculli, D. (1981). A stochastic cutting stock procedure: Cutting rolls of insulating tape. *Management Science*, 27(8), 946-952.

Sweeney, P. and Haessler, R.W. (1990). One-dimensional cutting stock decisions for rolls with multiple quality grades. *European Journal of Operational Research*, 44, 224-231.

Wäscher, G., Haussner, H., and Schumann, H. (2007). An improved topology of cutting and packing problems. *European Journal of Operational Research*, 183, 1109-1130.